

# Introducing Agile Development (XP) into a Corporate Webmaster Environment — An Experience Report

Matt Ganis and David Leip  
International Business Machines  
17 Skyline Drive  
Hawthorne, NY 10532  
ganis@us.ibm.com, leip@us.ibm.com

Fred Grossman and Joe Bergin  
School of Computer Science and  
Information Systems, Pace University  
1 Martine Avenue  
White Plains, NY 10606  
grossman@pace.edu, berginf@pace.edu

## Abstract

*In this paper we examine the challenges of moving a large, traditionally waterfall-driven organization toward an agile development methodology. In particular we address the problems/successes we encountered in our attempts to adhere to the 12 practices of Extreme Programming given: 1) this was a first attempt with Agile techniques and in particular extreme programming and 2) an organization that while supportive, is still driven by its own set of governance processes which seem to be in contradiction to extreme programming. We discuss how we prepared our development and customer teams in order to achieve an agile mindset along with the issues we encountered in our attempts to mesh IBM's internal processes with our implementation of Extreme Programming. Finally we address some of the lessons learned as revealed by our final project retrospective and our future plans using agile techniques*

## 1. Introduction

The IBM corporate web portal (<http://www.ibm.com>) was officially launched in May of 1994. Since that time, all application development has been performed using a typical waterfall methodology. As with any development organization, we are asked to cope with changing or inaccurate customer requirements coupled with frequent changes in direction and scope – all with the expectation of delivering a completed product within a

predetermined (by the customer/stakeholder) timeframe.

Our decision to move to an agile methodology was driven mainly by our desire to cope with the difficulty we faced in preparing well formed and complete requirements in a time frame that our customer required. Often there was enough knowledge of what was needed to allow us to move forward with some design and development without having to wait for a complete picture to be fully understood. A similar issue arose with projects involving multiple stakeholders. In some cases we had situations where the stakeholders could agree on the top five requirements, but there was lengthy discussion needed to obtain an agreement on the complete set of requirements. The result was that nothing was being built and no value was being delivered. Agile methodologies appeared to be a good solution to these issues. Budgets, planning and funding are always a challenge, thus the concept of having working code at the end of short release cycles was appealing not only to us as a development organization, but to our customer as well.

Our webmaster team is multinational, spanning several continents; however in order to simplify the introduction of agile development and to minimize the risk, the XP team for the project was limited to co-located team members. While we have a webmaster staff of over 20 professionals, they perform a variety of roles -- developer, XML technologist, standards/compliance and project management. The XP team consisted of 6 (of our 8) developers with varying ranges of skill and experience. For the purposes of the Planning Games, estimation and pairing our XP team worked as two separate

development skill groups, (Java and XML/XSL) because the skills were quite different. None of the developers possessed both skills, and it was unlikely we were going to turn XSL/DTD developers into Java developers and vice-versa. So each story had separate estimates one for Java and one for XML. Some of the Customer stories fell neatly into just one of these two functional categories, so one of the two sizing would be zero, but others spanned both Java and XML/XSL. The Java and XML/XSL developers operated as a whole team and participated together in the daily standup meetings and worked in the same XP team room. We feel that there might be a need to also include other skill groups, like web graphic designers for example, as part of the XP team to drive better end-to-end agility in the future

The average experience level on our agile team ranged from 3 to 5 years of professional experience (with one developer having over 10 years of professional development experience) and all had no prior experience with extreme programming.

All application development is performed by the Corporate Webmaster team. Functional strategy and direction is provided by our customer, an internal IBM organization (reporting to a different vice-president from the webmaster team). Our faculty partners from Pace University (Grossman and Bergin) served as XP coaches and facilitators – an extension of the development team. This paper is an experience report as written from the perspective of the development team and our coaches.

## 2. Importance of [www.ibm.com](http://www.ibm.com)

IBM's presence on the World Wide Web is supported in a highly distributed manner. Organizations within IBM traditionally have had their own web sites and have operated them in an autonomous manner. While the corporate webmasters staff is responsible for the enforcement of site standards, the operation of these satellite sites was left to the individual organizations. The IBM corporate portal is essentially the root of a large web presence that directs (or links) users to more and more specific sites within IBM.

While the corporate portal has a number of components running within it, the main function is to provide a landing page and links to the greater IBM. As main entry point for all traffic to IBM's websites, we see a significant volume of traffic, typically ranging from 40,000 to 50,000 hits per minute and peaking at about 60,000 hits per minute on an average week.

As a root node, the uptime and availability of the corporate portal is of paramount concern to us. As a result, we are cautious with any new application to the portal. For availability, we run our website across three geographically dispersed locations to ensure availability and response time[5]. This means that our applications and site are triply redundant which is driven not only by availability, but by capacity demands as well. The result is an availability record of 100.000% uptime for the last 4 years.

Our customer's goal was to have us develop a set of modules for our homepage to support a concept called "task based navigation". In short, the links on our pages were to be derived by querying a search engine and then displayed as part of the site navigation. As development proceeded, the scope shifted (or expanded) to include a new system we call Query Manager that would enable the customer to manage and fine-tune their queries before moving them into a production environment. All of this was to be supported for four countries, each with its own set of localized/translated pages and individual queries. At the same time, the complete look-and-feel of the IBM web presence was changing for nine countries (later expanded to the 84 countries we support) encompassing roughly 300,000 files (including html, css, graphics, etc).

As was stated previously, our corporate web portal is distributed over three major complexes, but we also support a test/qualification site as well that we refer to as our staging environment. The Query Manager functionality runs within the staging environment to validate queries and functionality, and when all data is correct, we publish the data from staging into our production environment (in this case essentially we replicate the staging database) to three redundant databases within our production environment.

### 3. Introduction to Agile

As we stated earlier, our desire to look at agile methodologies was driven primarily by our need to better deal with incomplete or highly volatile requirements. We initially met once a week in a study group, to discuss a different aspect of Extreme Programming and share what we learned with each other. While there were some pockets of agile activity inside IBM, we were unsuccessful at locating a coach from within the company (there has since been a surge in interest within IBM and several large, informal agile interest groups formed). In hindsight, the choice of Extreme Programming was probably driven more by the fact that there is a wealth of information about it rather than a fact based decision that it was the best fit of the agile methodologies for our organization.

Not having any formal experience with agile techniques, we approached the Dean of the School of Computer Science at Pace University (<http://www.csis.pace.edu>) through IBM's Centers for Advanced Studies<sup>i</sup> looking for experienced practitioners. Working with Dr. Fred Grossman and Dr. Joe Bergin as coaches, our development team and extended customer team underwent a series of training exercises using, among other things, a technique called "Extreme Construction" developed by Bergin and Grossman [2].

#### 3.1. Extreme Construction

Extreme Construction works as a way to both introduce the agile values, principles and the practices of Extreme Programming and the consequences of not performing those practices. It is fun, educational, and is also a great way to start to build a team spirit in an industrial project. In fact, exercises like this are a very effective way to bring a team into a cohesive whole in a short time at the beginning of a project, as the interactions are both fun and personal.

Extreme Construction tries to teach the agile values and principles and the basics of XP in a full day. All of the XP practices are included in at least skeletal form. The goal is not to perform the practices perfectly, but to try to do so while the process is monitored with an eye to determining

what goes wrong and how non-compliance with the practices leads to problems with the result.

The instructor/coach divides the participants into teams of about ten individuals. Two are selected to play the customer role, two the tracker role and the rest are developer/builders. A large quantity of arts and crafts materials of the kind used in kindergarten are available. The customers specify some artifact that they would like built by describing features of it on 3 by 5 inch cards. Small, independent features are best. The overall project is chosen with some instructor guidance so that there is enough complexity to ensure that some problems are likely to occur, but simple enough that the project is feasible.

Story cards are written continuously throughout the process by the customers and submitted to the developers to estimate (in ideal minutes). Estimates are written on the cards, and the cards are given back to the customers.

Iterations in the game last a fixed amount of time from 20 to 40 minutes. The team gives the customer a number, called the velocity, which is what the team thinks it can do in the current iteration in story points, where a story point in this game is equivalent to an ideal estimation minute. In the first iteration a wise team will give a small fraction (say one third) of the total available developer minutes. In 30 minute iteration with four developers, you have two developer-hours (real time) available, but it would be unwise to promise that much work. Estimates are just estimates, of course, and things happen. No team works at 100 percent efficiency over even short periods. And what were thought to be independent stories turn out not to be. Later stories will also require re-work, even destruction of carefully crafted artifacts built earlier. Such is the reality of development in the modern era. If stories are too big or complex, they are split into smaller stories or reduced in scope. The customer chooses the highest value stories available whose sum of estimates doesn't exceed the velocity. This becomes the basis of the iteration.

During the construction phase, developers work in pairs to build each story, integrating it into the whole as they go. The instructor (and trackers) should make sure that a pair doesn't divide the work up equally, but really works on it together. The only individual work is prototyping

(called spiking in XP) used to learn how to estimate or build something, but such spikes are not integrated into the build. At every moment everyone tries to "do the simplest thing that could possibly work" to build the story at hand.

The trackers keep track of whether the stories in the current iteration can all be completed, and at the earliest point that it is known not to be the case, the trackers inform the customer who then reduces the work in the iteration by pulling stories for later development. On the other hand, if work will complete early, the team goes to the customer for additional work. Customers accept or reject the work as it is completed, and somewhat more formally at the end of an iteration.

During an iteration, the customers respond to questions from the developers on what is wanted. We normally require that the developers provide a quick sketch of the feature before they begin construction (test first), and these are validated with the customer and then used after construction to verify that what was built conformed to what the customer said was wanted. We (Bergin and Grossman) consider this a weak (yet effective) form of test first development and are looking for a stronger alternative.

The major rule, however, is that the customers don't build or estimate and the developers don't specify or choose what to build. Both the instructor/coaches and trackers watch out for this. We are also strict about preventing coercion from either the customers or the developers. A common form of coercion here and in the real world is the customer demanding lower estimates or larger velocity.

Each iteration ends at a certain time whether the stories are completed or not. We value some stories not started with the others completed over many started and many not completed. Only then can you say for sure how much you can do in an iteration. The velocity for the next iteration is the same as you actually completed in the last (yesterday's weather) unless team size changes or the elapsed time is different. In these cases you just take a simple ratio. After a few iterations the estimates can lose their relation to actual time altogether and become abstract story points. As long as estimation is consistent, even if not accurate, you get a good sense about the rate (hence cost) of development that business

stakeholder can use to determine whether they are getting good value for the effort.

Note that what we have been describing here has been the construction game. However most of it also fairly applies to the XP process itself. The major difference is that the normal XP team creates software, not physical models, and of course, the elapsed time is much greater.

Any exercise of this type requires a period of reflection and discussion of the outcome. The time required for this is at least one sixth of the total time, and more if possible. In a classroom situation this discussion can continue later as the course delves deeper into the ideas and practices. However, the instructor must be in a position to know which deviations from practice caused which problems. In one of our sessions, planning the build became a bottleneck when a person who normally serves as a manager tried to over-manage the process top down. Moreover, he sat in a corner when this occurred, making it impossible for those on the outer edges of the quarter circle to participate. As a consequence, too little parallel work occurred, and the team ran out of time in the iteration. One team also tried to do an all in one design that was then later invalidated by a story they had not yet seen, causing much wasted work and a lot of rework. A very common problem is the developers thinking that they understand a story from a few words on the card and then making a lot of invalid assumptions about customer "needs" that they don't validate, even though the customer is only a few feet from them.

In the Extreme Construction game that we used to begin the training at IBM, the teams constructed a castle (complete with a moat and dragon) and a school playground. What we have observed is that the extreme construction game participants consistently behave in much the same manner as real software development XP teams. They want to do the best job that they can for the customer. In the early stages of the construction game (the first iteration) they will add "bells and whistles" that they believe will make the product better without asking the customer if she wants to pay for these at this time. They will design for expected additions and extensions without any supporting stories. They will design on the fly, sometimes alone, other times as a group. All of this in spite of proper coaching and customer on site.

They will start to pair, but quickly split off to work alone. Even worse, they will form larger ad hoc design groups (sometimes including all of the developers), and then some subset will build the design. This often does not involve interaction with the customer who is always on site. They will start out with test-first development, but quickly abandon this in order to get all the stories built before the iteration cycle deadline. They typically will integrate the built stories at the very end of the iteration so that the customer does not have time to properly test and accept them. The purpose of the exercise, of course, is not to perform perfectly the first time, but to provide a vehicle for examining the relationship between the process and the outcomes.

We have found that in the 4 to 6 hours of the construction game, the players continue to mimic much of the behavior of a development team in their first XP project. Most of the practices were sporadically followed, usually only after prodding by the coaches. However, at the end of the exercise, the majority of the participants understood the synergistic value of following all the practices, and expressed a commitment to do better on the next project. We have observed similar attitudes during the retrospectives of “real” XP development projects. Our observations, albeit limited, also have shown that teams that have played the game will follow more of the XP practices in their first XP project and will better understand their synergistic value than teams that have not. When we have both trained the teams using the Extreme Construction game and coached them in their first XP project, references to the game construction project frequently were used to make points and remind the team member of the XP practices and values.

An important aspect of this training was that we involved not only our development team, but our customer and several members of upstream providers of information to our project. This turned out to be critical later in the project, not because these providers were participating in our project, but because they could now understand our new mode of operating. This made communicating with these organizations easier and facilitated changes when our customer provided stories that caused a change to our data providers.

Once we trained the staff, it was time to start our project. We didn’t feel we were ready since practices such as test driven development were foreign to us and we weren’t familiar enough with the available tools. We made a conscious decision to attempt to master tools such as JUnit and HTMLUnit during the development process by setting our velocities low in the beginning to accommodate the learning curve. This is discussed further in [4]. Since this was our first attempt at an agile project, we also decided to attempt XP by the book. This meant we would not eliminate any of the practices but strive to implement them all.

#### 4. Project and Observations

An important aspect of this training was that we involved not only our development team, but our customer and several members of upstream providers of information to our project. This turned out to be critical later in the project, not because these providers were participating in our project, but because they could now understand our new mode of operating. This made communicating with these organizations easier and facilitated changes when our customer provided stories that caused a change to our data providers.

Once we trained the staff, it was time to start our project. We didn’t feel we were ready since practices such as test driven development were foreign to us and we weren’t familiar enough with the software. We made a conscious decision to attempt to learn things like JUnit and HTMLUnit during the development process by setting our velocities low in the beginning to allow for the learning curve. Since this was our first attempt at an agile project, we also decided to attempt XP by the book. This meant we would not eliminate any of the practices[1] but strive to implement them all. To explain our experiences with the basic practices of XP, we’ve broken the various practices down into three categories: Customer related      Teaming/Teamwork      and Design/Development.

On November 4, 2004 we officially launched our code into our public production site. This new application allows pages within our corporate portal to be dynamically generated by fine tuning a series of search engine queries that allow an

end-user to quickly and efficiently find the information they are looking for. There is also an internal component of this system for managing those queries that is used by our site architecture team that was part of this code deploy. All of this needed to be implemented within a new site wide redesign that involved the changing/updating of over 300,000 pages, graphics and layouts in nine languages. .

#### **4.1. Customer related Observations**

Being able to work in an environment of rapid feedback is fundamental to success with extreme programming or any agile technique. In our case, having involved our customer team early in our training sessions, they became acutely aware of the need for rapid feedback to questions during a development cycle. We were able to convince them to physically move their offices into our location to better facilitate communication with the development team. This proved to be an enormous benefit to the project, but a non-trivial cost to the customer organization. Having our customer on site meant that we were able to include them in all discussions related to the development, including our daily stand-up meetings. Unfortunately, one issue we ran into was our customer slipping back into business-as-usual and attempting to manage not just the stories for the project, but the resource as well. Our daily standup meetings turned from a development status/concern meeting to one in which the developers were reporting and justifying time spent on various aspects of the project. This led us back down a confrontational path between developer and customer. As a consequence, the developers stopped reporting issues/problems for fear of having the customer get upset. This was mainly a problem with one of the customer representatives. During the planning games when all the customer representatives were present, this was not as obvious. In fact, the “chief” customer met with the coaches and expressed concern that the developers were not following the XP practices. He wanted to know what he could do to help. The coaches told him to tell the developers that it was important to the customer that the XP practices be followed because he believed that this would enhance the quality of the process and the product. He did that and it was successful in changing the way the team functioned. XP practices improved.

Planning games were scheduled at the start of each of our two week iterations. Clearly the customer was comfortable with an iterative approach to planning the project. Several times during the project they changed directions, once dramatically, and reprioritized stories to focus on their critical needs. One issue we had with our planning games in the beginning was that we had too many people involved. The customer, along with their staffs and the development team and their management made for quite large planning meetings. While the open/honest communication with healthy, they would tend to get off track and at times, management/team leaders would get involved in sizing stories with the development staff, leading to incorrect estimates. Once the team understood this was a problem, estimates became more accurate as the developers became more comfortable with producing their own timelines and the customer came to understand the concept of a developer’s velocity.

#### **4.2. Team related Observations**

Early on during the inception of our project, when we were assembling the development team for this effort, we underestimated the number of Java developers we would need for this project. As a result we borrowed two former developers from our customer’s organization and had them temporarily join us as part of this newly forming team. In hindsight, we believe this disruption to the team caused some tensions, particularly with pairing and perhaps with collective code ownership. As the project progressed, and the team began to gel, these issues became less-and-less of a problem. As development team began to trust one another, and to acknowledge the strengths and weaknesses of its members, the team achieved a very strong cohesive nature. One thing we didn’t have time to come to a full agreement on was coding standards for the team. This was difficult to accomplish in the “heat of battle”, so while there were rudimentary standards in place, more work is needed in this area.

#### **4.3. Design/Development Observations**

Our team was able to easily adjust to creating small releases based on simple designs. Obviously this involved the principle of continuous integration. Our code is developed

using IBM's Web Sphere Application Development tool (WSAD) which is based on the eclipse framework. Doing frequent builds was somewhat of a problem for the team for two reasons: lack of experience with code management and lack of a clear understanding of a build process. We attempted to solve the latter problem with a set of automated build scripts, but like the test driven development, the developers never really had the time to get comfortable with this. As development progressed, the code management problem became less of an issue, but even during the last iteration, builds were being scheduled and delivered at predetermined times as opposed to nightly (or more frequently).

Working with our customer, we came to agreement that would produce builds on a set schedule or as stories were completed. To help make this easier, we dedicated two servers to our effort: an Integration Server and a "production Server". The Integration server was used by the developers to produce our regularly scheduled builds. The basic premise was that this server, while functional, was always under construction. At the end of each iteration, we installed a golden release of code that was properly integrated onto the production server, which was assumed to have the latest, fully functional build on it. This allowed our customers to demo our work to their peers, while not being hindered by development recycles or testing.

In the spirit of adhering to the XP practices, we attempted to implement a test first framework for the project. As we mentioned earlier, we knew our team wasn't prepared for some aspects of agile programming, and not having skills in automated unit testing software like JUnit, Cactus and HtmlUnit was a big (unprepared) area for us. We spent several weeks, while in development, attempting to get Test Driven Development to work for us with limited success. A testing fixture, HtmlFixture[4], now public, was developed by one of the authors to ease this problem, but it was little used. This is not a condemnation of the process, but an artifact of our schedule coupled with the lack of experience with the tools. We believe the code launch was successful and relatively bug free due to the fact that we also implement part of IBM's development process and went through 1 to 2 months of Function, Application Integration and

User acceptance testing before the product went live.

One important point that needs to be articulated is the use of a system metaphor. During our development cycle the team really never had a metaphor for the project, nor could we find a use for it. However, during the projects final retrospective it was clear that the development team lacked a clear understanding of the end-to-end product that we were being asked to deliver. It became clear that one of the things we lacked was an architecture that the group agreed upon and worked toward. In hindsight, we believe a proper use of a system metaphor could have helped tremendously in this area.

Finally, in the same vein as the system metaphor, a key role that seemed to be missing from our team was a system architect that understood how all of the pieces need to fit together, i.e., someone with high-level knowledge of external components we needed to interface with.

One other issue that was a concern to us was the deployment of this new application. Unlike shrink-wrapped application development, we are not done when the application development and testing are complete. We have no solution until we deploy into our large production environment, which means having the entire infrastructure in place, and operation teams in a position to support it. Working in an agile mode meant we didn't have the same sort of advance notice to interlock deployment dates (and capacity sizings) for the new application with our operations team until closer to our planned deployment date. This put an enormous strain on our operations team and clearly this an area we need to work on in the future, to better integrate development and deployment together into an agile approach.

## 5. Conclusions

Overall we report a highly successful experience with extreme programming. The success of any project can be measured in many ways: the number of defects, the timeliness, and the usefulness and of course the customer satisfaction. Through our use of extreme programming, we were able to achieve all of these goals. We believe this was due to several factors.

Early customer involvement was absolutely critical to our success, not from a pure requirements perspective, but having the customer understand and be part of the development process.

Early education was also a key factor to our success. It was clear that educating not only the team and the customer was important, but educating those groups that are outside the immediate team was critical as well. Again, it all came down to communications and awareness within the other groups. We also found that having highly knowledgeable coaches proved to be invaluable to our success. They not only help the team keep true to the methodology, but were valuable as arbiters between the old methodology mindset and the new methodology mindset.

We needed to create a better vision/architecture that would be constantly reviewed and updated by both the development team and the customer. This lack of a true vision almost caused us to not deliver our project. Certain architectural assumptions were made during one of our iterations that were later discovered to be inaccurate as we approached our launch date. A clearer architectural model that could be shared with our counterparts would have eliminated this problem (or brought it to light sooner).

## 6. Acknowledgements

We would like to thank our IBM development team which consisted of: Govind Nishar, Tom Hawkins, Joe Lopatka, Katie Dang, Chris Wong, Lisa Intervino and Kapil Gupta. Special thanks to

our IBM customer Mike Moran and his team for agreeing to try something new and bearing with us through this learning process as well as executive management, Darryl Turner, Susan Watson, Lee Dierdorff and John Rosato for giving us the latitude and support, to explore XP. Finally, special thanks to Robert Biddle our “shepherd” for this paper for providing valuable advice and suggestions.

## 7. References

- [1] K. Beck, *Extreme programming explained: embrace change*. 1999, Boston, MA: Addison-Wesley Longman Publishing Co., Inc.
- [2] J. Bergin, "eXtreme Construction-An XP Game". 2005:  
<http://csis.pace.edu/~bergin/extremeconstruction/index.html>.
- [3] J. Bergin, "HTMLFixture", 2005:  
<http://fitnesse.org/FitNesse.HtmlFixture>
- [4] F. Grossman, et al. "One XP experience: introducing agile (XP) software development into a culture that is willing but not ready". in *Conference of the Centre For Advanced Studies on Collaborative Research*. 2004. Markham, Ontario, Canada: IBM Press.
- [5] N. Yan, N., K. Gupta, and D. Leip, "The Use of Open-source software in the IBM Corporate Portal". IBM Systems Journal, 2005. 44(2): p. 419-425.